

Software Engineer — Backend & Platform [Take Home Test]

Option C: Multi-Tenant Config & Feature Flag Service

Time Expectation: 8 – 10 hours **Complexity:** 8/10

Note: Please make sure to only submit the take home test via GitHub and provide the url.

Forethought

We believe that the true essence of backend and platform engineering extends far beyond writing CRUD APIs and running `terraform apply`. In crafting this take-home exercise, we've thoughtfully designed it to reflect the authentic challenges and experiences a backend & platform engineer would encounter when building and operating production systems in a day-to-day environment.

As practicing engineers ourselves, we understand that no one can be an expert in every cloud service or infrastructure pattern. The reality of our profession is that we constantly navigate new territories — unfamiliar GCP services, novel deployment strategies, and complex system design trade-offs — through research, experimentation, and collaboration. This exercise is designed to honor that reality.

We embrace the modern development landscape where AI-powered tools like GitHub Copilot, ChatGPT, and Claude have become valuable allies in our development process. We strongly encourage you to leverage these tools, just as you would in your daily work. Your ability to effectively research, learn, implement & deliver solutions matters more to us than any pre-existing knowledge.

The exercise deliberately spans both backend development and platform/infrastructure engineering — from API design and data modeling to containerization, CI/CD, observability, and cloud infrastructure. This breadth isn't meant to overwhelm, but rather to give you the space to demonstrate your problem-solving approach and engineering mindset across the full spectrum. We're interested in seeing how you make architectural decisions, handle trade-offs, and document your thinking process — rather than seeing the perfect solution.

When evaluating your submission, we'll focus on your engineering approach — how you structure your solution, the clarity of your code, the thoughtfulness of your documentation, and the reasoning behind your technical decisions. We care about seeing your authentic problem-solving process rather than a perfect implementation.

Take this opportunity to show us how you think, learn, and create as a backend & platform engineer. Document your journey, explain your choices, and don't hesitate to make pragmatic decisions about where to focus your efforts. We're looking forward to understanding your unique perspective and approach to engineering through this exercise. We're not looking for a perfect solution!

Overview

Design and implement a **Multi-Tenant Configuration & Feature Flag Service** that allows multiple applications/tenants to manage feature flags and runtime configuration. The service should support percentage-based rollouts, environment-scoped configs, audit trails, and real-time config distribution. Then deploy the entire system on GCP with infrastructure-as-code, a blue-green (or canary) deployment strategy, and production-grade observability.

This task tests your ability to design multi-tenant data models, implement deterministic rollout algorithms, build secure APIs, manage infrastructure-as-code, deploy with zero-downtime strategies, and operate cloud-native systems on GCP — with a true 50/50 split between backend engineering and platform/DevOps.

Business Context

You're building an internal platform service that multiple applications within the organization will use to manage feature flags and runtime configuration. Think of it as a simplified LaunchDarkly or Unleash — a centralized service that lets teams:

1. Create and manage feature flags per application
2. Roll out features gradually using percentage-based targeting
3. Scope configurations to specific environments (dev, staging, production)
4. Audit every change for compliance and debugging
5. Distribute config changes to clients in real-time

The service needs to be multi-tenant (each application is a tenant), highly available, and deployed on GCP with proper infrastructure and operational tooling.

Functional Requirements

Backend (50%)

Tenant & Environment Management

- `POST /api/v1/tenants` — Register a new tenant (application)
- Each tenant has environments: `development`, `staging`, `production`
- API key authentication per tenant (each tenant gets a unique API key)

Feature Flag CRUD

- `POST /api/v1/tenants/{id}/flags` — Create a feature flag with name, description, type (boolean, string, number), and default value
- `GET /api/v1/tenants/{id}/flags` — List flags with filtering by environment and status (active/archived)
- `PUT /api/v1/tenants/{id}/flags/{flag_key}` — Update flag: toggle on/off, change rollout percentage, update targeting rules, modify default value
- `DELETE /api/v1/tenants/{id}/flags/{flag_key}` — Soft-delete (archive) a flag

Flag Evaluation Engine

- `POST /api/v1/evaluate` — Evaluate flags for a given user/context
 - Request body: `{ tenant_id, environment, user_id, context: { ... } }`
 - Returns evaluated flag values for that user
- `POST /api/v1/evaluate/bulk` — Bulk evaluate all active flags for a user context in a single request
- **Percentage rollouts must be deterministic:** the same `user_id` must always receive the same flag value for a given flag (use consistent hashing — e.g., hash of `flag_key + user_id` mapped to a 0-100 range)
- Support flag types: boolean (on/off), string (variant selection), and percentage-based gradual rollout

Audit & History

- Every flag change is automatically audit-logged: who changed it, what changed, when, and the previous value
- `GET /api/v1/tenants/{id}/flags/{flag_key}/history` — View chronological change history for a flag
- Audit records are immutable (append-only)

Real-Time Distribution (Bonus)

- SSE (Server-Sent Events) or WebSocket endpoint for clients to subscribe to flag changes for their tenant + environment
- When a flag is toggled or updated, subscribed clients receive the change in real-time

Platform & Infrastructure (50%)

Infrastructure-as-Code

- Use Terraform or Pulumi to provision all GCP resources:
 - Compute (Cloud Run, GKE, or Compute Engine)
 - Database (Cloud SQL PostgreSQL)
 - Caching (Memorystore for Redis — for caching flag evaluations)
 - Networking (VPC, subnets, firewall rules if applicable)
 - IAM roles and service accounts (least-privilege)
 - Secret Manager (for storing database credentials and API keys)
 - Cloud Monitoring (alert policies, notification channels)
- Modular, reusable structure with clear variable definitions and environment separation

Containerization

- Multi-stage Docker build optimized for production (small image, non-root user, health check)
- `docker-compose` for local development with all dependencies (API + PostgreSQL + Redis)

Deployment Strategy

- Implement **blue-green or canary deployment** on GCP:
 - Cloud Run: traffic splitting between revisions
 - GKE: rolling update with readiness/liveness probes
 - Or load balancer-based traffic shifting
- Document your choice of strategy and how rollback works

CI/CD Pipeline

- GitHub Actions pipeline with:
 - Lint → Test → Build container → Push to Artifact Registry → Deploy with traffic splitting
 - Separate workflows or stages for staging and production
 - Automated rollback on health check failure (bonus)

Observability Stack

- **Structured JSON logging** with request tracing (correlation IDs across all API calls)
- **Custom metrics** exported to Cloud Monitoring:

- Flag evaluation latency (p50, p95, p99)
- Evaluations per second (by tenant)
- Error rates (by tenant and endpoint)
- Cache hit/miss ratio
- **Cloud Monitoring dashboard** with the key metrics above
- **Alert policies** for:
 - Error rate spikes (>5% over 5-minute window)
 - Evaluation latency exceeding threshold
 - Service health check failures

Security

- API key authentication for tenants (keys stored hashed, not in plain text)
 - Rate limiting per tenant (to prevent noisy-neighbor problems)
 - All secrets managed via GCP Secret Manager (not environment variables in plain text)
 - Database credentials rotated via Secret Manager (bonus)
-

Testing Requirements

- **Unit tests** for the flag evaluation engine — especially percentage rollouts, deterministic hashing, and type handling
 - **Integration tests** for tenant isolation (verify one tenant cannot access another's flags) and environment scoping
 - **Load test script** (e.g., k6, Artillery, or autocannon) demonstrating the evaluation endpoint under concurrent load — document throughput and latency results
 - Tests should run in CI
 - Document your testing strategy: what you chose to test, why, and what you'd add with more time
-

Technical Specifications

- **Language:** Node.js/TypeScript preferred, Python acceptable
- **Framework:** NestJS recommended for this scope (modules, guards, interceptors help with multi-tenancy), Express/Fastify also fine
- **Database:** PostgreSQL via Cloud SQL (primary data store)
- **Cache:** Redis via Memorystore (for caching evaluated flag results)
- **laC:** Terraform preferred, Pulumi acceptable
- **Cloud:** GCP (required)
- **Container:** Docker (required)

Suggested Time Allocation

Phase	Time
System design, data modeling, IaC planning	1.5 – 2 hrs
Tenant management + feature flag CRUD	2 – 2.5 hrs
Flag evaluation engine + audit logging	1.5 – 2 hrs
Docker + IaC + CI/CD + blue-green deployment	2 – 2.5 hrs
Testing + observability + documentation	1.5 – 2 hrs

Expected Deliverables

1. **Source code in a Git repository**
 2. **README.md** with:
 - Setup instructions for local development
 - Architecture overview and system design decisions
 - Choice of technologies and reasoning (especially GCP services, caching strategy, deployment approach)
 - Database schema and data flow diagrams
 - API documentation with example requests/responses
 - Flag evaluation algorithm explanation (how percentage rollouts work)
 - Infrastructure architecture and deployment strategy
 - Assumptions made and trade-offs considered
 - Future improvements and scaling strategies
 - Testing strategy, load test results, and what you chose to test (and why)
 3. **Database schema and migrations**
 4. **Docker configuration** for local development
 5. **Infrastructure-as-Code files** (Terraform/Pulumi) with environment separation
 6. **CI/CD pipeline configuration** (GitHub Actions)
 7. **Deployed application URL on GCP**
-

Evaluation Criteria

- **Backend Engineering (25%):** Multi-tenant data modeling, flag evaluation engine correctness, API design, audit trail implementation, code structure
 - **Platform & DevOps (25%):** IaC quality, GCP service selection & reasoning, blue-green/canary deployment, Docker best practices, secret management
 - **Observability (15%):** Structured logging, custom metrics, dashboards, alerting — can you operate what you build?
 - **Testing (15%):** Flag evaluation correctness tests, tenant isolation tests, load testing, CI integration
 - **Documentation (10%):** Clear architecture docs, algorithm explanations, trade-off reasoning, setup instructions that actually work
 - **Decision Making (10%):** Why you chose what you chose — GCP services, caching strategy, deployment approach, framework choices, trade-offs
-

Submission

- Share a GitHub/GitLab repository link to asif@bizscout.com
 - Ensure the repository is public or provide access to the above email if private
 - Include the deployed GCP application URL in your README
-

Notes

- **Focus on getting a working implementation** rather than a perfect architecture
 - Document any shortcuts taken due to time constraints
 - Feel free to use any libraries, AI tools, agents, workflows or ANYTHING on the internet that speeds up development
 - **Prioritize functionality, testability & readability** over perfect code
 - GCP offers a free tier and \$300 in free credits for new accounts — use them
 - Some parts are intentionally kept vague for you to make appropriate assumptions — be rational about it and document them
 - The real-time distribution (SSE/WebSocket) feature is a bonus — implement it only if you have time after the core requirements are solid
 - If you have any questions, feel free to reach out to asif@bizscout.com
-

What We're Looking For

- With the internet at your fingertips, *how fast can you think → learn → execute & deliver and repeat the cycle*
 - How you split your brain between application code and infrastructure
 - Multi-tenant system design — do you understand data isolation, noisy-neighbor problems, and tenant-scoped access control?
 - GCP service selection — do you pick the right tool for the job and can you explain why?
 - Deterministic algorithms — can you implement consistent hashing for percentage rollouts?
 - Zero-downtime deployment — do you understand blue-green/canary and how rollback works?
 - Can you deploy and operate what you build, not just write it?
 - Observability as a first-class concern, not an afterthought
 - Clean API design with proper data flow and error handling
 - Practical problem-solving within time constraints
 - Clear documentation of your approach, decisions, and trade-offs
-

Good Luck, see you at the finish line!

Engineering Team

Words of wisdom from Asif:

Don't overthink, over-complicate, or over-engineer.

How do you eat an elephant? Bite by Bite or Byte by Byte!?